Module 17

Formal Relational Query Languages/2

Milav Dabgar

www.milav.in

March 17, 2026

Module Recap

- Relational Algebras and its Operations

Module Objectives

Goals

- To understand formal calculus-based query language through relational algebra

Module Outline

- Tuple Relational Calculus (Overview only)

Module Outline

- Tuple Relational Calculus (Overview only)
- Domain Relational Calculus (Overview only)

Module Outline

- Tuple Relational Calculus (Overview only)
- Domain Relational Calculus (Overview only)
- Equivalence of Algebra and Calculus

Formal Relational Query Language

- Relational Algebra

Formal Relational Query Language

- Relational Algebra
  - Procedural and Algebra based

Formal Relational Query Language

- Relational Algebra
  - Procedural and Algebra based
- Tuple Relational Calculus

Formal Relational Query Language

- Relational Algebra
  - Procedural and Algebra based
- Tuple Relational Calculus
  - Non-Procedural and Predicate Calculus based

Formal Relational Query Language

- Relational Algebra
  - Procedural and Algebra based
- Tuple Relational Calculus
  - Non-Procedural and Predicate Calculus based
- Domain Relational Calculus

Formal Relational Query Language

- Relational Algebra
  - Procedural and Algebra based
- Tuple Relational Calculus
  - Non-Procedural and Predicate Calculus based
- Domain Relational Calculus
  - Non-Procedural and Predicate Calculus based

Predicate Logic

Predicate Logic

Overview

- Predicate Logic or Predicate Calculus is an extension of Propositional Logic or Boolean Algebra.

Predicate Logic

Overview

- Predicate Logic or Predicate Calculus is an extension of Propositional Logic or Boolean Algebra.
- It adds the concept of predicates and quantifiers to better capture the meaning of statements that cannot be adequately expressed by propositional logic.

Predicate Logic

Overview

- Predicate Logic or Predicate Calculus is an extension of Propositional Logic or Boolean Algebra.
- It adds the concept of predicates and quantifiers to better capture the meaning of statements that cannot be adequately expressed by propositional logic.
- Tuple Relational Calculus and Domain Relational Calculus are based on Predicate Calculus.

Predicate

Definition

- Consider the statement, 'x is greater than 3'. It has two parts. The first part, the variable x, is the subject of the statement. The second part, 'is greater than 3', is the predicate. It refers to a property that the subject of the statement can have.

Predicate

Definition

- Consider the statement, 'x is greater than 3'. It has two parts. The first part, the variable x, is the subject of the statement. The second part, 'is greater than 3', is the predicate. It refers to a property that the subject of the statement can have.
- The statement 'x is greater than 3' can be denoted by $P(x)$ where P denotes the predicate 'is greater than 3' and x is the variable.

Predicate

Definition

- Consider the statement, 'x is greater than 3'. It has two parts. The first part, the variable x, is the subject of the statement. The second part, 'is greater than 3', is the predicate. It refers to a property that the subject of the statement can have.
- The statement 'x is greater than 3' can be denoted by $P(x)$ where P denotes the predicate 'is greater than 3' and x is the variable.
- The predicate P can be considered as a function. It tells the truth value of the statement $P(x)$ at x. Once a value has been assigned to the variable x, the statement $P(x)$ becomes a proposition and has a truth or false value.

Predicate

Definition

- Consider the statement, 'x is greater than 3'. It has two parts. The first part, the variable x, is the subject of the statement. The second part, 'is greater than 3', is the predicate. It refers to a property that the subject of the statement can have.
- The statement 'x is greater than 3' can be denoted by $P(x)$ where P denotes the predicate 'is greater than 3' and x is the variable.
- The predicate P can be considered as a function. It tells the truth value of the statement $P(x)$ at x. Once a value has been assigned to the variable x, the statement $P(x)$ becomes a proposition and has a truth or false value.
- In general, a statement involving n variables $x_1, x_2, x_3, \ldots, x_n$ can be denoted by $P(x_1, x_2, x_3, \ldots, x_n)$. Here P is also referred to as n-place predicate or a n-ary predicate.

Quantifiers

In predicate logic, predicates are used alongside quantifiers to express the extent to which a predicate is true over a range of elements. Using quantifiers to create such propositions is called quantification. There are two types of quantifiers:

- Universal Quantifier

Quantifiers

In predicate logic, predicates are used alongside quantifiers to express the extent to which a predicate is true over a range of elements. Using quantifiers to create such propositions is called quantification. There are two types of quantifiers:

- Universal Quantifier
- Existential Quantifier

Universal Quantifier

Universal Quantification: Mathematical statements sometimes assert that a property is true for all the values of a variable in a particular domain, called the domain of discourse.

- Such a statement is expressed using universal quantification.

Universal Quantifier

Universal Quantification: Mathematical statements sometimes assert that a property is true for all the values of a variable in a particular domain, called the domain of discourse.

- Such a statement is expressed using universal quantification.
- The universal quantification of $P(x)$ for a particular domain is the proposition that asserts that $P(x)$ is true for all values of x in this domain.

Universal Quantifier

Universal Quantification: Mathematical statements sometimes assert that a property is true for all the values of a variable in a particular domain, called the domain of discourse.

- Such a statement is expressed using universal quantification.
- The universal quantification of $P(x)$ for a particular domain is the proposition that asserts that $P(x)$ is true for all values of x in this domain.
- The domain is very important here since it decides the possible values of x.

Universal Quantifier

Universal Quantification: Mathematical statements sometimes assert that a property is true for all the values of a variable in a particular domain, called the domain of discourse.

- Such a statement is expressed using universal quantification.
- The universal quantification of $P(x)$ for a particular domain is the proposition that asserts that $P(x)$ is true for all values of x in this domain.
- The domain is very important here since it decides the possible values of x.
- Formally, The universal quantification of $P(x)$ is the statement '$P(x)$ for all values of x in the domain'.

Universal Quantifier

Universal Quantification: Mathematical statements sometimes assert that a property is true for all the values of a variable in a particular domain, called the domain of discourse.

- Such a statement is expressed using universal quantification.
- The universal quantification of $P(x)$ for a particular domain is the proposition that asserts that $P(x)$ is true for all values of x in this domain.
- The domain is very important here since it decides the possible values of x.
- Formally, The universal quantification of $P(x)$ is the statement '$P(x)$ for all values of x in the domain'.
- The notation $\forall P(x)$ denotes the universal quantification of $P(x)$. Here $\forall$ is called the universal quantifier. $\forall P(x)$ is read as 'for all x $P(x)$'.

Universal Quantifier

Universal Quantification: Mathematical statements sometimes assert that a property is true for all the values of a variable in a particular domain, called the domain of discourse.

- Such a statement is expressed using universal quantification.
- The universal quantification of $P(x)$ for a particular domain is the proposition that asserts that $P(x)$ is true for all values of x in this domain.
- The domain is very important here since it decides the possible values of x.
- Formally, The universal quantification of $P(x)$ is the statement '$P(x)$ for all values of x in the domain'.
- The notation $\forall P(x)$ denotes the universal quantification of $P(x)$. Here $\forall$ is called the universal quantifier. $\forall P(x)$ is read as 'for all x $P(x)$'.
- Example: Let $P(x)$ be the statement '$x + 2 > x$'. What is the truth value of the statement $\forall x P(x)$?

Universal Quantifier

Universal Quantification: Mathematical statements sometimes assert that a property is true for all the values of a variable in a particular domain, called the domain of discourse.

- Such a statement is expressed using universal quantification.
- The universal quantification of $P(x)$ for a particular domain is the proposition that asserts that $P(x)$ is true for all values of x in this domain.
- The domain is very important here since it decides the possible values of x.
- Formally, The universal quantification of $P(x)$ is the statement '$P(x)$ for all values of x in the domain'.
- The notation $\forall P(x)$ denotes the universal quantification of $P(x)$. Here $\forall$ is called the universal quantifier. $\forall P(x)$ is read as 'for all x $P(x)$'.
- Example: Let $P(x)$ be the statement '$x + 2 > x$'. What is the truth value of the statement $\forall x P(x)$?
- Solution: As $x + 2$ is greater than x for any real number, so $P(x) \equiv T$ for all x or $\forall x P(x) \equiv T$.

Existential Quantifier

Existential Quantification: Some mathematical statements assert that there is an element with a certain property. Such statements are expressed by existential quantification. Existential quantification can be used to form a proposition that is true if and only if $P(x)$ is true for at least one value of x in the domain.

- Formally, the existential quantification of $P(x)$ is the statement 'There exists an element x in the domain such that $P(x)$'.

Existential Quantifier

Existential Quantification: Some mathematical statements assert that there is an element with a certain property. Such statements are expressed by existential quantification. Existential quantification can be used to form a proposition that is true if and only if $P(x)$ is true for at least one value of x in the domain.

- Formally, the existential quantification of $P(x)$ is the statement 'There exists an element x in the domain such that $P(x)$'.
- The notation $\exists P(x)$ denotes the existential quantification of $P(x)$. Here $\exists$ is called the existential quantifier. $\exists P(x)$ is read as 'There is atleast one such x such that $P(x)$'.

Existential Quantifier

Existential Quantification: Some mathematical statements assert that there is an element with a certain property. Such statements are expressed by existential quantification. Existential quantification can be used to form a proposition that is true if and only if $P(x)$ is true for at least one value of x in the domain.

- Formally, the existential quantification of $P(x)$ is the statement 'There exists an element x in the domain such that $P(x)$'.
- The notation $\exists P(x)$ denotes the existential quantification of $P(x)$. Here $\exists$ is called the existential quantifier. $\exists P(x)$ is read as 'There is atleast one such x such that $P(x)$'.
- Example: Let $P(x)$ be the statement '$x > 5$'. What is the truth value of the statement $\exists x P(x)$?

Existential Quantifier

Existential Quantification: Some mathematical statements assert that there is an element with a certain property. Such statements are expressed by existential quantification. Existential quantification can be used to form a proposition that is true if and only if $P(x)$ is true for at least one value of x in the domain.

- Formally, the existential quantification of $P(x)$ is the statement 'There exists an element x in the domain such that $P(x)$'.
- The notation $\exists P(x)$ denotes the existential quantification of $P(x)$. Here $\exists$ is called the existential quantifier. $\exists P(x)$ is read as 'There is atleast one such x such that $P(x)$'.
- Example: Let $P(x)$ be the statement '$x > 5$'. What is the truth value of the statement $\exists x P(x)$?
- Solution: $P(x)$ is true for all real numbers greater than 5 and false for all real numbers less than 5. So $\exists x P(x) \equiv T$.

Tuple Relational Calculus

Tuple Relational Calculus

Definition

TRC is a non-procedural query language, where each query is of the form

$$\{t \mid P(t)\}$$

- where t = resulting tuples, $P(t)$ = known as predicate and these are the conditions that are used to fetch t. $P(t)$ may have various conditions logically combined with OR ($\vee$), AND ($\wedge$), NOT($\neg$).

Tuple Relational Calculus

Definition

TRC is a non-procedural query language, where each query is of the form

$$\{t \mid P(t)\}$$

- where t = resulting tuples, $P(t)$ = known as predicate and these are the conditions that are used to fetch t. $P(t)$ may have various conditions logically combined with OR ($\vee$), AND ($\wedge$), NOT($\neg$).
- It also uses quantifiers:

Tuple Relational Calculus

Definition

TRC is a non-procedural query language, where each query is of the form

$$\{t \mid P(t)\}$$

- where t = resulting tuples, $P(t)$ = known as predicate and these are the conditions that are used to fetch t. $P(t)$ may have various conditions logically combined with OR ($\vee$), AND ($\wedge$), NOT($\neg$).
- It also uses quantifiers:
  - $\exists t \in r(Q(t))$ = 'there exists' a tuple in t in relation r such that predicate $Q(t)$ is true.

Tuple Relational Calculus

Definition

TRC is a non-procedural query language, where each query is of the form

$$\{t \mid P(t)\}$$

- where t = resulting tuples, $P(t)$ = known as predicate and these are the conditions that are used to fetch t. $P(t)$ may have various conditions logically combined with OR ($\vee$), AND ($\wedge$), NOT($\neg$).
- It also uses quantifiers:
  - $\exists t \in r(Q(t))$ = 'there exists' a tuple in t in relation r such that predicate $Q(t)$ is true.
  - $\forall t \in r(Q(t))$ = $Q(t)$ is true 'for all' tuples in relation r.

Tuple Relational Calculus

Definition

TRC is a non-procedural query language, where each query is of the form

$$\{t \mid P(t)\}$$

- where t = resulting tuples, $P(t)$ = known as predicate and these are the conditions that are used to fetch t. $P(t)$ may have various conditions logically combined with OR ($\vee$), AND ($\wedge$), NOT($\neg$).
- It also uses quantifiers:
  - $\exists t \in r(Q(t))$ = 'there exists' a tuple in t in relation r such that predicate $Q(t)$ is true.
  - $\forall t \in r(Q(t))$ = $Q(t)$ is true 'for all' tuples in relation r.
- $\{P \mid \exists S \in \text{Students and } (S.\text{CGPA} > 8 \wedge P.\text{name} = S.\text{sname} \wedge P.\text{age} = S.\text{age})\}$: returns the name and age of students with a CGPA above 8.

Predicate Calculus Formula

- a) Set of attributes and constants

Predicate Calculus Formula

- a) Set of attributes and constants
- b) Set of comparison operators: (e.g., $<, \leq, =, \neq, >, \geq$)

Predicate Calculus Formula

- a) Set of attributes and constants
- b) Set of comparison operators: (e.g., $<, \leq, =, \neq, >, \geq$)
- c) Set of connectives: and ($\wedge$), or ($\vee$), not ($\neg$)

Predicate Calculus Formula

- a) Set of attributes and constants
- b) Set of comparison operators: (e.g., $<, \leq, =, \neq, >, \geq$)
- c) Set of connectives: and ($\wedge$), or ($\vee$), not ($\neg$)
- d) Implication ($\Rightarrow$): $x \Rightarrow y$, if x is true, then y is true. $x \Rightarrow y \equiv \neg x \vee y$

Predicate Calculus Formula

- a) Set of attributes and constants
- b) Set of comparison operators: (e.g., $<, \leq, =, \neq, >, \geq$)
- c) Set of connectives: and ($\wedge$), or ($\vee$), not ($\neg$)
- d) Implication ($\Rightarrow$): $x \Rightarrow y$, if x is true, then y is true. $x \Rightarrow y \equiv \neg x \vee y$
- e) Set of quantifiers:

Predicate Calculus Formula

- a) Set of attributes and constants
- b) Set of comparison operators: (e.g., $<, \leq, =, \neq, >, \geq$)
- c) Set of connectives: and ($\wedge$), or ($\vee$), not ($\neg$)
- d) Implication ($\Rightarrow$): $x \Rightarrow y$, if x is true, then y is true. $x \Rightarrow y \equiv \neg x \vee y$
- e) Set of quantifiers:
  - ▷ $\exists t \in r(Q(t)) \equiv$ 'there exists' a tuple in t in relation r such that predicate $Q(t)$ is true

Predicate Calculus Formula

- a) Set of attributes and constants
- b) Set of comparison operators: (e.g., $<, \leq, =, \neq, >, \geq$)
- c) Set of connectives: and ($\wedge$), or ($\vee$), not ($\neg$)
- d) Implication ($\Rightarrow$): $x \Rightarrow y$, if x is true, then y is true. $x \Rightarrow y \equiv \neg x \vee y$
- e) Set of quantifiers:
  - $\exists t \in r(Q(t)) \equiv$ 'there exists' a tuple in t in relation r such that predicate $Q(t)$ is true
  - $\forall t \in r(Q(t)) \equiv Q$ is true 'for all' tuples t in relation r

TRC Example

Student Fname	Student Lname	Student Age	Student Course
David	Sharma	27	DBMS
Aaron	Lilly	17	JAVA
Sahil	Khan	19	Python
Sachin	Rao	20	DBMS
Varun	George	23	JAVA
Simi	Verma	22	JAVA

Q.1 Obtain the first name of students whose age is greater than 21.

Solution:

- $\{t.\text{Fname} \mid \text{Student}(t) \wedge t.\text{age} > 21\}$

TRC Example

Student Fname	Student Lname	Student Age	Student Course
David	Sharma	27	DBMS
Aaron	Lilly	17	JAVA
Sahil	Khan	19	Python
Sachin	Rao	20	DBMS
Varun	George	23	JAVA
Simi	Verma	22	JAVA

Q.1 Obtain the first name of students whose age is greater than 21.

Solution:

- $\{t.\text{Fname} \mid \text{Student}(t) \wedge t.\text{age} > 21\}$
- $\{t.\text{Fname} \mid t \in \text{Student} \wedge t.\text{age} > 21\}$

TRC Example

Student Fname	Student Lname	Student Age	Student Course
David	Sharma	27	DBMS
Aaron	Lilly	17	JAVA
Sahil	Khan	19	Python
Sachin	Rao	20	DBMS
Varun	George	23	JAVA
Simi	Verma	22	JAVA

Q.1 Obtain the first name of students whose age is greater than 21.

Solution:

- $\{t.\text{Fname} \mid \text{Student}(t) \wedge t.\text{age} > 21\}$
- $\{t.\text{Fname} \mid t \in \text{Student} \wedge t.\text{age} > 21\}$
- $\{t \mid \exists s \in \text{Student}(s.\text{age} > 21 \wedge t.\text{Fname} = s.\text{Fname})\}$

TRC Example (2)

Consider the relational schema `student(rollNo, name, year, courseId)` and `course(courseId, cname, teacher)`

- Q.2 Find out the names of all students who have taken the course name 'DBMS'.

TRC Example (2)

Consider the relational schema `student(rollNo, name, year, courseId)` and `course(courseId, cname, teacher)`

- Q.2 Find out the names of all students who have taken the course name 'DBMS'.
  - $\{t \mid \exists s \in \text{student}\ \exists c \in \text{course}\ (s.\text{courseId} = c.\text{courseId} \wedge c.\text{cname} =' \text{DBMS}' \wedge t.\text{name} = s.\text{name})\}$

TRC Example (2)

Consider the relational schema `student(rollNo, name, year, courseId)` and `course(courseId, cname, teacher)`

- Q.2 Find out the names of all students who have taken the course name 'DBMS'.
  - $\{t \mid \exists s \in \text{student}\ \exists c \in \text{course}\ (s.\text{courseId} = c.\text{courseId} \wedge c.\text{cname} =' \text{DBMS}' \wedge t.\text{name} = s.\text{name})\}$
  - $\{s.\text{name} \mid s \in \text{student} \wedge \exists c \in \text{course}\ (s.\text{courseId} = c.\text{courseId} \wedge c.\text{cname} =' \text{DBMS}')\}$

TRC Example (2)

Consider the relational schema `student(rollNo, name, year, courseId)` and `course(courseId, cname, teacher)`

- Q.2 Find out the names of all students who have taken the course name 'DBMS'.
  - $\{t \mid \exists s \in \text{student}\ \exists c \in \text{course}\ (s.\text{courseId} = c.\text{courseId} \wedge c.\text{cname} =' \text{DBMS}' \wedge t.\text{name} = s.\text{name})\}$
  - $\{s.\text{name} \mid s \in \text{student} \wedge \exists c \in \text{course}\ (s.\text{courseId} = c.\text{courseId} \wedge c.\text{cname} =' \text{DBMS}')\}$
- Q.3 Find out the names of all students and their `rollNo` who have taken the course name 'DBMS'.

TRC Example (2)

Consider the relational schema `student(rollNo, name, year, courseId)` and `course(courseId, cname, teacher)`

- Q.2 Find out the names of all students who have taken the course name 'DBMS'.
  - $\{t \mid \exists s \in \text{student}\ \exists c \in \text{course}\ (s.\text{courseId} = c.\text{courseId} \wedge c.\text{cname} =' \text{DBMS}' \wedge t.\text{name} = s.\text{name})\}$
  - $\{s.\text{name} \mid s \in \text{student} \wedge \exists c \in \text{course}\ (s.\text{courseId} = c.\text{courseId} \wedge c.\text{cname} =' \text{DBMS}')\}$
- Q.3 Find out the names of all students and their `rollNo` who have taken the course name 'DBMS'.
  - $\{s.\text{name}, s.\text{rollNo} \mid s \in \text{student} \wedge \exists c \in \text{course}\ (s.\text{courseId} = c.\text{courseId} \wedge c.\text{cname} =' \text{DBMS}')\}$

TRC Example (2)

Consider the relational schema `student(rollNo, name, year, courseId)` and `course(courseId, cname, teacher)`

- Q.2 Find out the names of all students who have taken the course name 'DBMS'.
  - $\{t \mid \exists s \in \text{student}\ \exists c \in \text{course}\ (s.\text{courseId} = c.\text{courseId} \wedge c.\text{cname} =' \text{DBMS}' \wedge t.\text{name} = s.\text{name})\}$
  - $\{s.\text{name} \mid s \in \text{student} \wedge \exists c \in \text{course}\ (s.\text{courseId} = c.\text{courseId} \wedge c.\text{cname} =' \text{DBMS}')\}$
- Q.3 Find out the names of all students and their `rollNo` who have taken the course name 'DBMS'.
  - $\{s.\text{name}, s.\text{rollNo} \mid s \in \text{student} \wedge \exists c \in \text{course}\ (s.\text{courseId} = c.\text{courseId} \wedge c.\text{cname} =' \text{DBMS}')\}$
  - $\{t \mid \exists s \in \text{student}\ \exists c \in \text{course}\ (s.\text{courseId} = c.\text{courseId} \wedge c.\text{cname} =' \text{DBMS}' \wedge t.\text{name} = s.\text{name} \wedge t.\text{rollNo} = s.\text{rollNo})\}$

TRC Example (3)

Consider the following relations:

`Flights(flno, from, to, distance, departs, arrives)`

`Aircraft(aid, aname, cruisingrange)`

`Certified(eid, aid)`

`Employees(eid, ename, salary)`

Q.4. Find the eids of pilots certified for Boeing aircraft.

RA

$$\Pi_{\text{eid}}(\sigma_{\text{aname}='\text{Boeing}'}(\text{Aircraft} \bowtie \text{Certified}))$$

TRC

- $\{C.\text{eid} \mid C \in \text{Certified} \wedge \exists A \in \text{Aircraft}(A.\text{aid} = C.\text{aid} \wedge A.\text{aname} =' \text{Boeing}')\}$

TRC Example (3)

Consider the following relations:

`Flights(flno, from, to, distance, departs, arrives)`

`Aircraft(aid, aname, cruisingrange)`

`Certified(eid, aid)`

`Employees(eid, ename, salary)`

Q.4. Find the eids of pilots certified for Boeing aircraft.

RA

$$\Pi_{\text{eid}}(\sigma_{\text{aname}='\text{Boeing}'}(\text{Aircraft} \bowtie \text{Certified}))$$

TRC

- $\{C.\text{eid} \mid C \in \text{Certified} \wedge \exists A \in \text{Aircraft}(A.\text{aid} = C.\text{aid} \wedge A.\text{aname} =' \text{Boeing}')\}$
- $\{T \mid \exists C \in \text{Certified}\ \exists A \in \text{Aircraft}(A.\text{aid} = C.\text{aid} \wedge A.\text{aname} =' \text{Boeing}' \wedge T.\text{eid} = C.\text{eid})\}$

TRC Example (4)

Consider the following relations:

`Flights(flno, from, to, distance, departs, arrives)`

`Aircraft(aid, aname, cruisingrange)`

`Certified(eid, aid)`

`Employees(eid, ename, salary)`

Q.5. Find the names and salaries of certified pilots working on Boeing aircrafts.

RA

$$\Pi_{\text{ename, salary}}(\sigma_{\text{aname}='\text{Boeing}'}(\text{Aircraft} \bowtie \text{Certified} \bowtie \text{Employees}))$$

TRC

$$\{P \mid \exists E \in \text{Employees}\ \exists C \in \text{Certified}\ \exists A \in \text{Aircraft}(A.\text{aid} = C.\text{aid} \wedge A.\text{aname} =' \text{Boeing}' \wedge E.\text{eid} =$$

TRC Example (5)

Consider the following relations:

`Flights(flno, from, to, distance, departs, arrives)`

`Aircraft(aid, aname, cruisingrange)`

`Certified(eid, aid)`

`Employees(eid, ename, salary)`

Q.6 Identify the flights that can be piloted by every pilot whose salary is more than $\$100,000$. (Hint: The pilot must be certified for at least one plane with a sufficiently large cruising range.)

- $\{F.\text{flno} \mid F \in \text{Flights} \wedge \exists A \in \text{Aircraft} \exists C \in \text{Certified} \exists E \in \text{Employees}(A.\text{cruisingrange} > F.\text{distance} \wedge A.\text{aid} = C.\text{aid} \wedge E.\text{salary} > 100,000 \wedge E.\text{eid} = C.\text{eid})\}$

Safety of Expressions

Warning on Safety

- It is possible to write tuple calculus expressions that generate infinite relations.

Safety of Expressions

Warning on Safety

- It is possible to write tuple calculus expressions that generate infinite relations.
- For example, $\{t \mid \neg t \in r\}$ results in an infinite relation if the domain of any attribute of relation r is infinite.

Safety of Expressions

Warning on Safety

- It is possible to write tuple calculus expressions that generate infinite relations.
- For example, $\{t \mid \neg t \in r\}$ results in an infinite relation if the domain of any attribute of relation r is infinite.
- To guard against the problem, we restrict the set of allowable expressions to **safe expressions**.

Safety of Expressions

Warning on Safety

- It is possible to write tuple calculus expressions that generate infinite relations.
- For example, $\{t \mid \neg t \in r\}$ results in an infinite relation if the domain of any attribute of relation r is infinite.
- To guard against the problem, we restrict the set of allowable expressions to **safe expressions**.
- An expression $\{t \mid P(t)\}$ in the tuple relational calculus is safe if every component of t appears in one of the relations, tuples, or constants that appear in P.

Safety of Expressions

Warning on Safety

- It is possible to write tuple calculus expressions that generate infinite relations.
- For example, $\{t \mid \neg t \in r\}$ results in an infinite relation if the domain of any attribute of relation r is infinite.
- To guard against the problem, we restrict the set of allowable expressions to **safe expressions**.
- An expression $\{t \mid P(t)\}$ in the tuple relational calculus is safe if every component of t appears in one of the relations, tuples, or constants that appear in P.
- NOTE: this is more than just a syntax condition.

Safety of Expressions

Warning on Safety

- It is possible to write tuple calculus expressions that generate infinite relations.
- For example, $\{t \mid \neg t \in r\}$ results in an infinite relation if the domain of any attribute of relation r is infinite.
- To guard against the problem, we restrict the set of allowable expressions to **safe expressions**.
- An expression $\{t \mid P(t)\}$ in the tuple relational calculus is safe if every component of t appears in one of the relations, tuples, or constants that appear in P.
- NOTE: this is more than just a syntax condition.
- E.g. $\{t \mid t[A] = 5 \vee \text{true}\}$ is not safe - it defines an infinite set with attribute values that do not appear in any relation or tuples or constants in P.

Domain Relational Calculus

Domain Relational Calculus

Definition

- A non-procedural query language equivalent in power to the tuple relational calculus.

Domain Relational Calculus

Definition

- A non-procedural query language equivalent in power to the tuple relational calculus.
- Each query is an expression of the form:

$$\{\langle x_1, x_2, \ldots, x_n \rangle \mid P(x_1, x_2, \ldots, x_n)\}$$

Domain Relational Calculus

Definition

- A non-procedural query language equivalent in power to the tuple relational calculus.
- Each query is an expression of the form:

$$\{\langle x_1, x_2, \ldots, x_n \rangle \mid P(x_1, x_2, \ldots, x_n)\}$$

- $x_1, x_2, \ldots, x_n$ represent domain variables.

Domain Relational Calculus

Definition

- A non-procedural query language equivalent in power to the tuple relational calculus.
- Each query is an expression of the form:

$$\{\langle x_1, x_2, \ldots, x_n \rangle \mid P(x_1, x_2, \ldots, x_n)\}$$

- $x_1, x_2, \ldots, x_n$ represent domain variables.
- P represents a formula similar to that of the predicate calculus.

Equivalence of Algebra and Calculus

Equivalence of RA, TRC and DRC (1)

Equivalence of RA, TRC and DRC: Select Operation

Equivalence of RA, TRC and DRC: Project Operation

Equivalence of RA, TRC and DRC: Combining Operations

Equivalence of RA, TRC and DRC (2)

Union

R = (A, B, C) S = (A, B, C)

Relational Algebra: $r \cup s$

Tuple Calculus: $\{t \mid t \in r \vee t \in s\}$

Domain Calculus: $\{\langle a, b, c\rangle \mid \langle a, b, c\rangle \in r \vee \langle a, b, c\rangle \in s\}$

Equivalence of RA, TRC and DRC: Set Difference

Set Difference

R = (A, B, C) S = (A, B, C)

Relational Algebra: $r - s$

Tuple Calculus: $\{t \mid t \in r \wedge t \notin s\}$

Domain Calculus: $\{\langle a, b, c\rangle \mid \langle a, b, c\rangle \in r \wedge \langle a, b, c\rangle \notin s\}$

Equivalence of RA, TRC and DRC: Intersection

Equivalence of RA, TRC and DRC: Cartesian/Cross Product

Cartesian/Cross Product

R = (A, B) S = (C, D)

Relational Algebra: $r \times s$

Tuple Calculus: $\{t \mid \exists\, p \in r\ \exists\, q \in s\ (t[A] = p[A] \wedge t[B] = p[B] \wedge t[C] = q[C] \wedge t[D] = q[D])\}$

Domain Calculus: $\{\langle a, b, c, d\rangle \mid \langle a, b\rangle \in r \wedge \langle c, d\rangle \in s\}$

Equivalence of RA, TRC and DRC: Natural Join

Natural Join

R = (A, B, C, D) S = (B, D, E)

Relational Algebra: $r \bowtie s$

$$\Pi_{r.A,r.B,r.C,r.D,s.E}(\sigma_{r.B=s.B \wedge r.D=s.D}(r \times s))$$

Tuple Calculus: $\{t \mid \exists\, p \in r\ \exists\, q \in s\ (t[A] = p[A] \wedge t[B] = p[B] \wedge t[C] = p[C] \wedge t[D] = p[D] \wedge t[E] = q[E] \wedge p[B] = q[B] \wedge p[D] = q[D])\}$

Domain Calculus: $\{\langle a, b, c, d, e\rangle \mid \langle a, b, c, d\rangle \in r \wedge \langle b, d, e\rangle \in s\}$

Equivalence of RA, TRC and DRC: Division

Module Summary

Summary

- Introduced tuple relational and domain relational calculus

Slides used in this presentation are borrowed from `http://db-book.com/` *with kind permission of the authors.*
Edited and new slides are marked with 'PPD'.

Module Summary

Summary

- Introduced tuple relational and domain relational calculus
- Illustrated equivalence of algebra and calculus

Slides used in this presentation are borrowed from `http://db-book.com/` *with kind permission of the authors.*
Edited and new slides are marked with 'PPD'.